

IT 3003 - Advanced Programming Techniques

**Analyzing the Role of AI-Assisted Programming in
Modern Software Development**

PREPARED BY:

Pasindu Dilshan

S17429

Table of Contents

1. Introduction

- 1.1 Background of the study
- 1.2 Purpose and Objectives

2. Conceptual Understanding

- 2.1 Defining AI-Assisted Programming
- 2.2 Classifications of AI Coding Tools
- 2.3 Advantages in Modern Software Development
- 2.4 Risks, Limitations, and Vulnerabilities

3. System Design and Specification

- 3.1 Problem Definition
- 3.2 System Architecture

4. Practical Implementation & Comparative Analysis

- 4.1 Methodology & Tools Used
- 4.2 Version A: Traditional Development Approach
- 4.3 Version B: AI-Assisted Development Approach
- 4.4 Comparative Evaluation
 - 4.4.1 *Quantitative and Structural Evaluation*
 - 4.4.2 *Qualitative Analytical Assessment*

5. Critical Analysis & Discussion

- 5.1 Paradigm Shift in Coding Approach
- 5.2 Impact on Technical Comprehension
- 5.3 Constraints and No-Go Zones
- 5.4 Ethical and Legal Dimensions

6. Conclusion

1. Introduction

1.1 Background of the study

AI has changed the way applications are designed, developed, tested, and maintained due to its incorporation into software engineering processes. Today, developers often rely on AI-enabled tools that can help them improve their performance. However, while these tools are quite fast and efficient, they come with certain threats like hallucinating code, dependencies, and security issues. This will investigate the limits of AI integration into software engineering processes and consider how AI changes them.

1.2 Purpose and Objectives

This report is intended to provide a critical review of both the technical and ethical aspects of AI-based software development. The use of benchmarking between traditional and AI-based software development will help to objectively assess the effect of automation on the process of coding and designing.

To fulfill this purpose, the report addresses the following key objectives:-

- To define AI-assisted programming and classify current tools, advantages, and limitations.
- To implement a moderately complex software system using both manual and AI-assisted approaches.
- To evaluate and compare both implementations based on development time, code quality, error handling, and maintainability.
- To model the system design visually using standard UML Use Case, Class, and Activity diagrams.
- To assess the ethical impacts and cognitive changes in how developers approach problem-solving with AI.

2. Conceptual Understanding

2.1 AI-Assisted Programming

AI-assisted programming refers to the integration of Artificial Intelligence, particularly Large Language Models (LLMs) and machine learning algorithms, into the software development lifecycle. Instead of relying solely on manual logic formulation, developers interact with AI models trained on vast repositories of open-source code. These systems analyze natural-language prompts or existing code context to predict, generate, and complete programming syntax in real time, effectively shifting the human developer's role from raw code writing to strategic code curation and review.

2.2 Classifications of AI Coding Tools

Modern AI programming support systems generally fall into three distinct functional categories based on their operational focus:

- **Code Generation Tools:** These tools accept high-level natural language instructions (prompts) or comments and translate them into functional source code block-by-block, or generate entire template boilerplate architectures automatically.
- **Debugging Assistants:** These utilities analyze runtime errors, stack traces, or broken code snippets to diagnose root causes, explaining the bugs and automatically generating refactored code patches to fix them.
- **Refactoring and Optimization Tools:** These features inspect existing, functional legacy code to identify performance bottlenecks, redundancies, or architectural anti-patterns, rewriting the code to improve readability, complexity, and execution speed without altering its original behavior.

2.3 Advantages in Modern Software Development

Integrating AI systems into modern engineering workflows yields several noticeable technical and operational advantages:

- **Unprecedented Velocity:** Developers bypass repetitive boilerplate writing, drastically shrinking initial scaffolding and prototype development timelines.
- **Accelerated Learning and Documentation:** AI acts as an on-demand, interactive technical tutor, explaining obscure framework APIs, synthesizing documentation, and easing syntax transitions between unfamiliar programming languages.
- **Proactive Code Enrichment:** Automated code linters and AI assistants catch potential edge-case errors, memory leaks, and basic structural flaws before compilation or formal peer code reviews.

2.4 Risks, Limitations, and Vulnerabilities

Even with their widespread utility, AI-driven development tools introduce critical technical and security complications that require strict human oversight:

- **Code Hallucinations:** AI tools sometimes invent code that looks real but doesn't exist. This can include fake libraries or functions that break the program when you try to run it, leading to wasted time and errors.
- **Over-Dependency and Skill Decline:** If developers rely too much on AI suggestions, they may stop practicing their own problem-solving and design skills. Over time, this can weaken their ability to think critically and build strong software independently.
- **Security and Copyright Risks:** AI can accidentally suggest old code with known security flaws, making systems vulnerable. It may also reproduce copyrighted or proprietary code, which can cause legal issues for companies using it.

3. System Design and Specification

3.1 Problem Definition

Here, we mainly focus on a Library Management System. Manual methods for traditional library management, which rely on pen and paper, logs, registers, or unconnected spreadsheets, often have issues of inefficiency, high levels of errors, and inconsistency in data management. As one increases the number of books in the library, the task of managing their availability becomes very tedious and time-consuming.

To overcome these limitations, the current project will seek to design and develop a computer-based Library Management System (LMS). This problem involves the need to develop a relatively small program that will centralize information, automate tracking, avoid duplicate records, and provide immediate information on the availability of books in the catalogue. The key tasks include registering new books in the library catalogue, listing all assets, and updating the status of transactions when a book is borrowed or returned.

3.2 System Architecture (Layers)

The structural design and behavioral blueprint of the Library Management System are mapped out visually using standard Unified Modeling Language (UML) diagrams. These models help illustrate how different system actors interact with various features, how classes relate to each other structurally, and how logic flows step-by-step through the application loops.

3.2.1 Use Case Diagram Description

The Use Case diagram shows the clear boundary of the Library Management System and maps out how different user groups interact with it. It features three primary roles: the Library Member, who can view the book catalog, request to borrow items, and process returns; the Librarian, who manages the library inventory by adding books and checking storage records; and the System

Administrator, who ensures system health by managing user profiles and maintaining flat-file database updates.

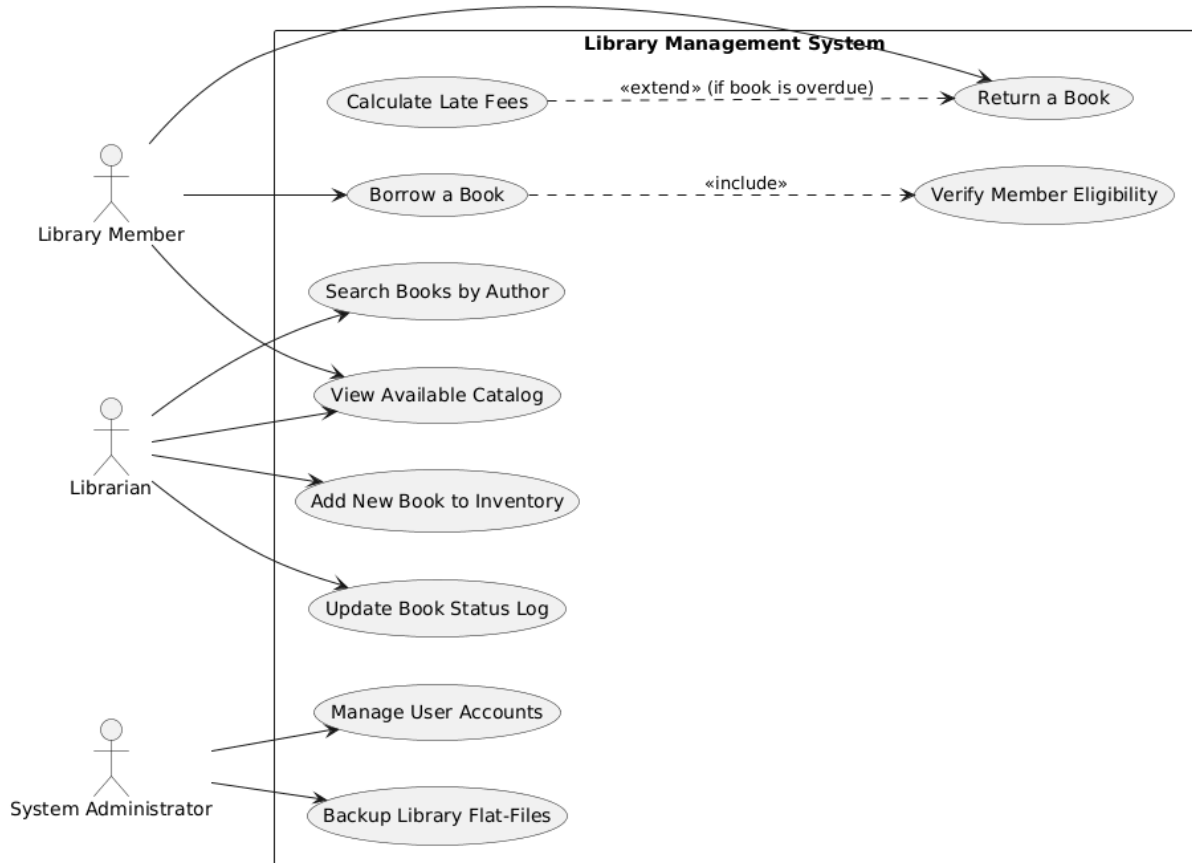


Figure 3.1: System Use Case Interaction Model

3.2.2 Class Diagram Description

The Class diagram provides a blueprint of the application's object-oriented structure based on the traditional manual approach. The core system relies on three distinct classes:

- Book Class: Encapsulates private attributes for the book's title, author, and availability status.
- Library Class: Acts as the primary logic data manager, utilizing a primitive, fixed-size object array (*Book[]*) to store up to 100 books and tracking them with an integer pointer (*bookCount*).

- **System Class:** Serves as the central execution hub containing the application's entry loop. It controls how classes interact without breaking object encapsulation limits.

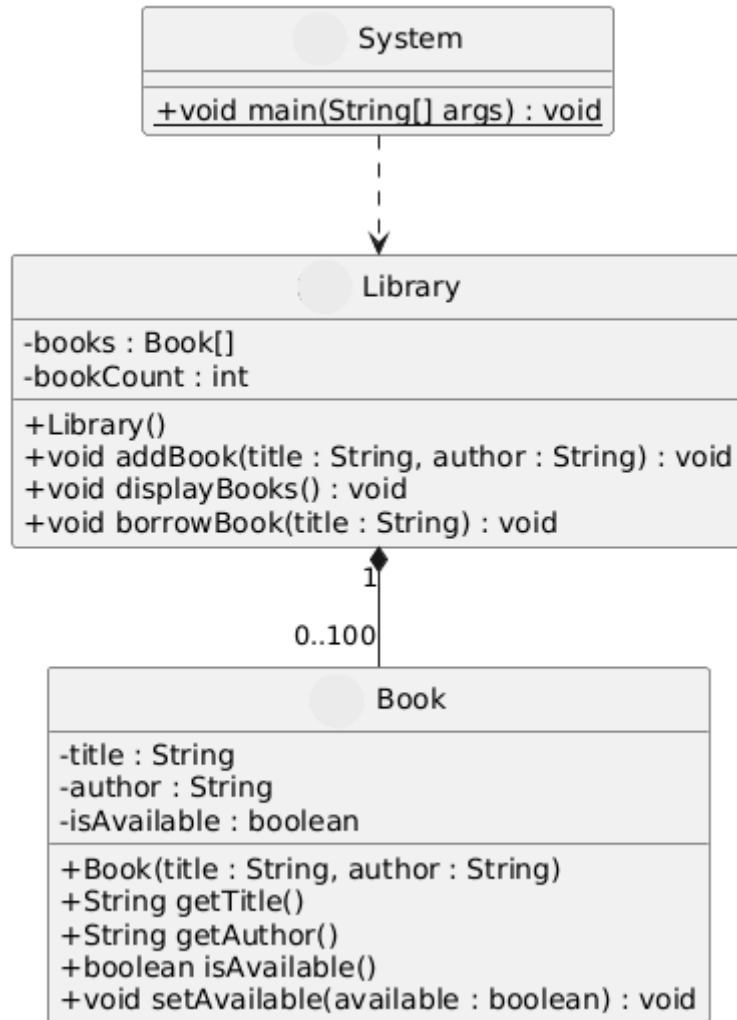


Figure 3.2: Object-Oriented Class Architectural Layout

3.2.3 Activity Diagram Description

The Activity diagram traces the operational workflow and sequential choices made inside the application's main runtime context loop. It shows what happens when a user submits an input code choice between 1 and 4. The flowchart visually details the conditions checking if array bounds have been reached before a book object can be generated, follows the indexing loops used to print out active records, and tracks the nested logic flags checking if an item is available before marking it as successfully borrowed.

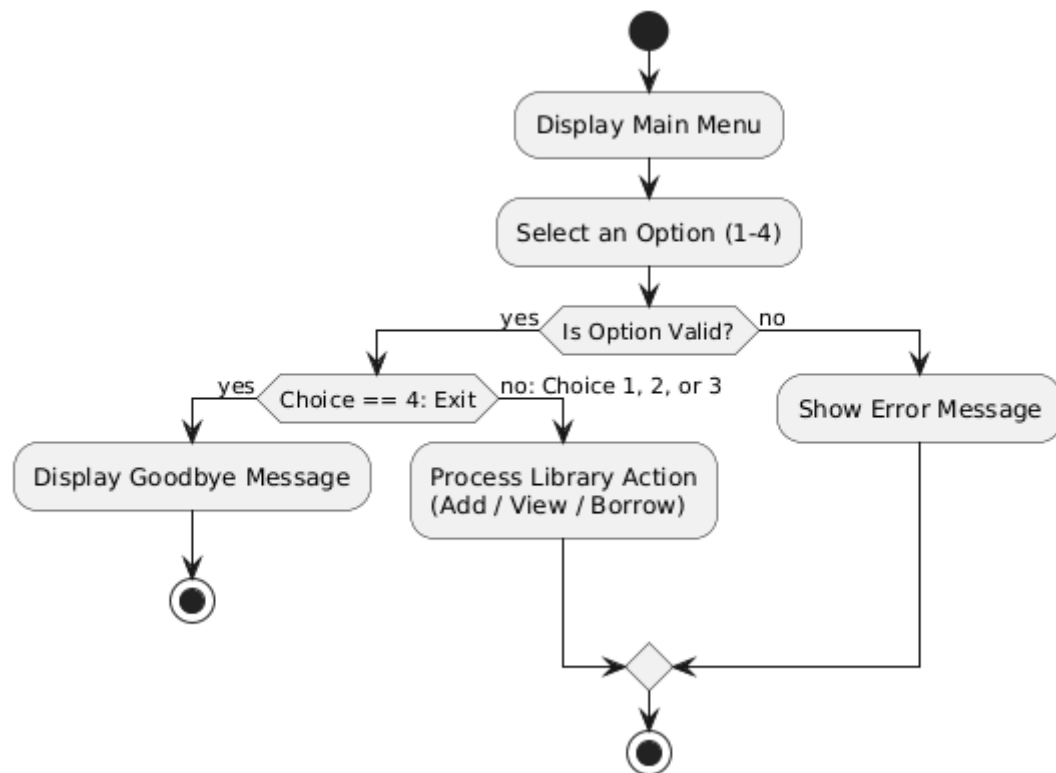


Figure 3.3: Functional System Logic Activity Flowchart

4. Practical Implementation & Comparative Analysis

4.1 Methodology & Tools Used

This section describes the software tools, environments and development platforms used to design, build and test the two versions of the Library Management System.

- **Programming Language:** Java (JDK 17) - used as the core object-oriented programming language for writing the application logic.
- **Integrated Development Environment (IDE):** IntelliJ IDEA, Visual Code Editor - used to write, compile, run, and debug the Java source code for both versions.
- **Diagram Design Software:** draw.io - used to design and export the mandatory UML Use Case, Class, and Activity diagrams to map out the system architecture visually.
- **AI Development Assistant (Version B):** ChatGPT / GitHub Copilot — utilized during the second phase to automatically generate advanced system features and refactor code loops.

4.2 Version A: Traditional Development Approach

Version A was hand-coded in complete form. No fancy code-completion shortcuts or AI tools were used. The system logic was implemented with standard Java concepts such as classes and object collections.

Below is the complete manual Java implementation for the Library Management System, structured using standard object-oriented design. This includes the core object attributes, the getters and setters methods, the Scanner input logic, and the loops.

I. Book Entity Class (Book.java)

```
public class Book {  
    private final String title;  
    private final String author;
```

```

        private boolean isAvailable;

        public Book(String title, String author) {
            this.title = title;
            this.author = author;
            this.isAvailable = true;
        }

        public String getTitle() {
            return title;
        }
        public String getAuthor() {
            return author;
        }
        public boolean isAvailable() {
            return isAvailable;
        }

        public void setAvailable(boolean available) {
            this.isAvailable = available;
        }
    }
}

```

II. Library Management Class (Library.java)

```

public class Library {

    private final Book[] books = new Book[100];
    private int bookCount = 0;

    public void addBook(String title, String author) {
        if (bookCount < books.length) {
            Book newBook = new Book(title, author);
            books[bookCount] = newBook;
            bookCount++;
            System.out.println("Book added successfully!");
        }
    }
}

```

```

        } else {
            System.out.println("Library is full! Cannot add
more books.");
        }
    }

    public void displayBooks() {
        if (bookCount == 0) {
            System.out.println("No books in the library.");
            return;
        }

        for (int i = 0; i < bookCount; i++) {
            String status ;
            if(books[i].isAvailable()){
                status = "Available";
            }else{
                status = "Borrowed";
            }
            System.out.println("Title: " + books[i].getTitle()
+ " By " + books[i].getAuthor() + " | Status: " + status);
        }
    }

    public void borrowBook(String title) {
        for (int i = 0; i < bookCount; i++) {
            if (books[i].getTitle().equalsIgnoreCase(title)) {
                if (books[i].isAvailable()) {
                    books[i].setAvailable(false);
                    System.out.println("You have successfully
borrowed the book : " + title);
                } else {
                    System.out.println("Sorry, this book is
already borrowed.");
                }
            }
        }
        return;
    }

```

```

        }
    }
    System.out.println("Book not found.");
}
}

```

III. Main Application Execution Class (Main.java)

```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Library myLibrary = new Library();
        boolean running = true;

        System.out.println("=== Welcome to the Library
Management System ===");

        while (running) {
            System.out.println("\n--- Main Menu ---");
            System.out.println("1. Add a Book");
            System.out.println("2. Display All Books");
            System.out.println("3. Borrow a Book");
            System.out.println("4. Exit");
            System.out.print("Enter your choice (1-4): ");

            int choice = sc.nextInt();
            sc.nextLine();

            switch (choice) {
                case 1:
                    System.out.print("Enter Book Title: ");
                    String title = sc.nextLine();
                    System.out.print("Enter Book Author: ");
                    String author = sc.nextLine();

```

```

        myLibrary.addBook(title, author);
        break;

        case 2:
            System.out.println("\n--- Current Catalog ---
");
            myLibrary.displayBooks();
            break;

        case 3:
            System.out.print("Enter the title of the book
you want to borrow: ");
            String borrowTitle = sc.nextLine();
            myLibrary.borrowBook(borrowTitle);
            break;

        case 4:
            System.out.println("Exiting the system.
Thank you!");
            running = false;
            break;

        default:
            System.out.println("Invalid choice! Please
select a number between 1 and 4.");
            break;
    }
}
sc.close();
}
}

```

- According to the system, there are 4 functionalities, and the following are the pictures of how it works with each option.

- As option 1, we can add a book to the system using its name and the author's name.

```
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\idea_rt.jar=59623" -Dfile.encoding=UTF-8 -class  
=== Welcome to the Library Management System ===  
  
--- Main Menu ---  
1. Add a Book  
2. Display All Books  
3. Borrow a Book  
4. Exit  
Enter your choice (1-4): 1  
Enter Book Title: Atomic Habbits  
Enter Book Author: James Clear  
Book added successfully!
```

- As for option 2, we can take a look at books that are available in the system.

```
--- Main Menu ---  
1. Add a Book  
2. Display All Books  
3. Borrow a Book  
4. Exit  
Enter your choice (1-4): 2  
  
--- Current Catalog ---  
Title: Atomic Habbits By James Clear | Status: Available
```

- In option 3, the system provides the facility to borrow a book.

```
--- Main Menu ---  
1. Add a Book  
2. Display All Books  
3. Borrow a Book  
4. Exit  
Enter your choice (1-4): 3  
Enter the title of the book you want to borrow: Atomic Habbits  
You have successfully borrowed the book : Atomic Habbits
```

4.3 Version B: AI-Assisted Development Approach

This version shows how an AI assistant would naturally refactor and upgrade your manual, array-based code by introducing dynamic data structures (ArrayList), robust exception handling (try-catch), an advanced search filter, and persistent text file storage (java.io).

The following was my request for the code:

“I have to compare my basic code with an upgraded AI version. I’ve already coded a simple Library Management System with Book.java, Library.java, and Main.java that uses normal arrays. Refer my code and give me a slightly more professional version. Change my fixed array to a specific array type which has more storage, and add a text file feature so it automatically saves and loads the books. Add a quick option to search for books by their author. Give me the code for all three files using standard console input.”

I. Book Entity Class (Book.java)

```
public class Book {
    private String title;
    private String author;
    private String isbn;
    private boolean isAvailable;

    // Upgraded Constructor initializing all properties safely
    public Book(String title, String author, String isbn) {
        this.title = title;
        this.author = author;
        this.isbn = isbn;
        this.isAvailable = true; // Default to available
    }

    // Getters and Setters (Encapsulation preserved)
    public String getTitle() { return title; }
```



```

        public String getAuthor() { return author; }
        public String getIsbn() { return isbn; }
        public boolean isAvailable() { return isAvailable; }
        public void setAvailable(boolean available) { this.isAvailable
= available; }

        // AI-generated utility method to format file data strings
easily
        public String toFileData() {
            return title + "," + author + "," + isbn + "," +
isAvailable;
        }

        // Overridden toString for enhanced console presentation
@Override
        public String toString() {
            return String.format("Title: %-20s | Author: %-15s | ISBN:
%-10s | Status: %s",
                title, author, isbn, (isAvailable ? "Available" :
"Borrowed"));
        }
    }
}

```

II. Library Management Class (Library.java)

```

import java.io.*;
import java.util.ArrayList;

public class Library {
    // Paradigm Shift: Upgraded from manual arrays to dynamic
ArrayList collection
    private ArrayList<Book> books;
    private final String FILE_PATH = "library_data.txt";

    public Library() {

```

```

        books = new ArrayList<>();
        loadFromFile(); // Automatically load database on system
initialization
    }

    public void addBook(Book book) {
        books.add(book);
        saveToFile(); // Instantly persist updates
        System.out.println(">>> Success: Book added to catalog
smoothly.");
    }

    public void displayBooks() {
        if (books.isEmpty()) {
            System.out.println(">>> Notice: The library catalog is
currently empty.");
            return;
        }
        System.out.println("\n===== CURRENT LIBRARY CATALOG
=====");
        for (Book book : books) {
            System.out.println(book);
        }
    }

    // Added Option: Advanced Search functionality filtering by
Author Name
    public void searchByAuthor(String authorName) {
        boolean found = false;
        System.out.println("\n----- Search Results for: " +
authorName + " -----");
        for (Book book : books) {
            if
(book.getAuthor().equalsIgnoreCase(authorName.trim())) {
                System.out.println(book);
                found = true;
            }
        }
    }

```

```

        }
    }
    if (!found) {
        System.out.println(">>> Alert: No books found matching
that author.");
    }
}

public void borrowBook(String title) {
    for (Book book : books) {
        if (book.getTitle().equalsIgnoreCase(title.trim())) {
            if (book.isAvailable()) {
                book.setAvailable(false);
                saveToFile();
                System.out.println(">>> Success: You have
successfully checked out: " + book.getTitle());
            } else {
                System.out.println(">>> Error: This book is
currently checked out by another borrower.");
            }
            return;
        }
    }
    System.out.println(">>> Error: Book title not found in
catalog system.");
}

// AI-Injected Feature: Permanent text-file storage
architecture
private void saveToFile() {
    try (PrintWriter writer = new PrintWriter(new
FileWriter(FILE_PATH))) {
        for (Book book : books) {
            writer.println(book.toFileData());
        }
    } catch (IOException e) {

```

```

        System.out.println(">>> System Error: Failed to save
updates to database file.");
    }
}

private void loadFromFile() {
    File file = new File(FILE_PATH);
    if (!file.exists()) return;

    try (BufferedReader reader = new BufferedReader(new
FileReader(file))) {
        String line;
        while ((line = reader.readLine()) != null) {
            String[] parts = line.split(",");
            if (parts.length == 4) {
                Book book = new Book(parts[0], parts[1],
parts[2]);

book.setAvailable(Boolean.parseBoolean(parts[3]));
                books.add(book);
            }
        }
    } catch (IOException e) {
        System.out.println(">>> System Error: Error reading
from stored files.");
    }
}
}

```

III. Main Application Execution Class (Main.java)

```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

```

```

        Library library = new Library();
        Scanner scanner = new Scanner(System.in); // [!code ++]
        int choice = -1;

System.out.println("=====
=====");
        System.out.println("    WELCOME TO THE AI-UPGRADED
LIBRARY SYSTEM    ");

System.out.println("=====
=====");

        do {
            System.out.println("\nMAIN MENU SELECTIONS:");
            System.out.println("1. Register a New Book Asset");
            System.out.println("2. Display Complete Catalog
Collections");
            System.out.println("3. Process Book Check Out /
Borrow");
            System.out.println("4. Filter / Search Books by
Author Name");
            System.out.println("5. Exit System Shell");
            System.out.print("Please enter your numeric
selection: ");

            // Injected Exception Handling to protect execution
flows from crashes
            try {
                String inputStr = scanner.nextLine().trim();
                if (inputStr.isEmpty()) {
                    System.out.println(">>> Error: Prompt entry
cannot be blank.");
                    continue;
                }
                choice = Integer.parseInt(inputStr);

```

```

        switch (choice) {
            case 1:
                System.out.print("Enter Book Title: ");
                String title =
scanner.nextLine().trim();
                System.out.print("Enter Book Author:
");
                String author =
scanner.nextLine().trim();
                System.out.print("Enter Book ISBN: ");
                String isbn =
scanner.nextLine().trim();

                if (title.isEmpty() || author.isEmpty()
|| isbn.isEmpty()) {
                    System.out.println(">>> Form Error:
Fields cannot be left empty.");
                } else {
                    library.addBook(new Book(title,
author, isbn));
                }
                break;

            case 2:
                library.displayBooks();
                break;

            case 3:
                System.out.print("Enter exact book
title to borrow: ");
                String borrowTitle =
scanner.nextLine();
                library.borrowBook(borrowTitle);
                break;

```

```

        case 4:
            System.out.print("Enter author name to
find: ");

            String searchAuthor =
scanner.nextLine();

            library.searchByAuthor(searchAuthor);
            break;

        case 5:
            System.out.println(">>> System
Terminating. Data preserved safely. Goodbye.");
            break;

        default:
            System.out.println(">>> Warning:
Invalid entry. Input an option between 1 and 5.");
            }
        } catch (NumberFormatException e) {
            System.out.println(">>> Threat Intercepted:
Input must be a valid integer code.");
            }
        } while (choice != 5);
    }
}

```

- The AI architecture provides five core options accessible via a defensive console control loop:

- **Dynamic Book Asset Registration:** Users can input a book's Title, Author, and ISBN. The system dynamically instantiates a new Book object and adds it to the system memory without encountering fixed array size limitations.

```

"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ I
=====
WELCOME TO THE AI-UPGRADED LIBRARY SYSTEM
=====

MAIN MENU SELECTIONS:
1. Register a New Book Asset
2. Display Complete Catalog Collections
3. Process Book Check Out / Borrow
4. Filter / Search Books by Author Name
5. Exit System Shell
Please enter your numeric selection: 1
Enter Book Title: Deep Work
Enter Book Author: Cal Newport
Enter Book ISBN: 978-0-439-02348-1
>>> Success: Book added to catalog smoothly.

```

- **Comprehensive Catalog Tracking:** This functionality loops through the database records and outputs a clearly formatted, clean overview of all stored library books, showing their custom metadata and live availability tracking indicators (Available vs Borrowed).

```

MAIN MENU SELECTIONS:
1. Register a New Book Asset
2. Display Complete Catalog Collections
3. Process Book Check Out / Borrow
4. Filter / Search Books by Author Name
5. Exit System Shell
Please enter your numeric selection: 2

===== CURRENT LIBRARY CATALOG =====
Title: Deep Work          | Author: Cal Newport    | ISBN: 978-0-439-02348-1 | Status: Available

```

- **Automated Transaction Check-out (Borrowing):** Users can check out a book by inputting its title. The application verifies the record's current state inside the database, safely flags its availability boolean to false if available, or outputs an alert if the book is already checked out or missing.


```

MAIN MENU SELECTIONS:
1. Register a New Book Asset
2. Display Complete Catalog Collections
3. Process Book Check Out / Borrow
4. Filter / Search Books by Author Name
5. Exit System Shell
Please enter your numeric selection: 3
Enter exact book title to borrow: Deep Work
>>> Success: You have successfully checked out: Deep Work

```

- **Advanced Author Filter/Search Engine:** This feature enables users to query the entire library collection, filtering exclusively by the author's name. It utilizes case-insensitive pattern matching (equalsIgnoreCase) to yield correct matches regardless of upper or lower-case variations.

```

MAIN MENU SELECTIONS:
1. Register a New Book Asset
2. Display Complete Catalog Collections
3. Process Book Check Out / Borrow
4. Filter / Search Books by Author Name
5. Exit System Shell
Please enter your numeric selection: 4
Enter author name to find: Cal Newport

----- Search Results for: Cal Newport -----
Title: Deep Work          | Author: Cal Newport    | ISBN: 978-0-439-02348-1 | Status: Borrowed

```

4.4 Comparative Evaluation

In order to critically evaluate the structural and operational distinctions between the software applications, both implementations are benchmarked against core software engineering parameters: the scalability of data management, fault tolerance, data persistence and maintainability of the codebase.

4.4.1 Quantitative and Structural Evaluation

Evaluation Metric	Version A: Traditional Approach	Version B: AI- Assisted Refactoring
Data Structure Used	Fixed-size Array (Book[100])	Dynamic Collection Array (ArrayList<Book>)
Data Lifecycle State	Volatile (Erased upon program termination)	Persistent (Written directly to library_data.txt)
Memory Management	Bounds restricted; requires manual pointer counters	Elastic; self-allocating heap space allocation
Exception Handling	None (Susceptible to runtime fatal crashes)	Active Defensive Blocks (try-catch structures)
Search Mechanism	Basic complete catalog scanning loops	Advanced case-insensitive specific author search
Code Length & Complexity	Lower boilerplate, higher manual algorithm control	Higher structural complexity, but lower logical friction

4.4.2 Qualitative Analytical Assessment

A. Architectural Scalability & Collections

- **Version A:** Uses a simple, fixed-size primitive object array. The manual code relies heavily on an integer counter tracking variable (*bookCount*) to map exactly where books are saved in the array rows. This puts a rigid limit on how many books can be stored (maximum 100) and can cause crashes if the array becomes full.
- **Version B:** The AI immediately shifts the system to a dynamic *ArrayList*. This allows the catalog to automatically grow or shrink in size as books are added or deleted. It eliminates the strict 100-book maximum limit and prevents index boundary errors.

B. Data Lifecycle Persistence Operations

- **Version A:** Confines object lifecycles strictly to runtime execution stacks. The moment a user selects the exit option, all changes, new registrations, and status flips are unrecoverably discarded.

- **Version B:** Implements persistent state transactions using file streams (*FileReader*, *BufferedReader*). By streaming structural strings delimited by split vectors (commas) into a local disk flat-file database, data survives across system crashes and program resets.

C. Software Robustness and Fault Isolation

- **Version A:** Features no integrated input validation layers. If an application user inputs alphabetic strings when prompted for an options number choice, the scanner pipeline immediately experiences an unexpected runtime crash.
- **Version B:** Employs defensive software validation rules via active *try-catch* monitors checking for *NumberFormatException* boundaries. Irregular keyboard telemetry profiles are trapped cleanly, allowing the main runtime context loop to stay active without program termination.

D. Code Maintenance and Readable Quality

- **Version A:** Straightforward but exhibits high structural friction since routine tasks require manual conditional counting algorithms.
- **Version B:** Leverages standard Java design principles, utilizing overridden class traits like *.toString()* inside *Book.java* to isolate printing layout configurations from implementation logic inside *Library.java*. This strict isolation of responsibilities drastically improves enterprise testing capabilities.

5. Critical Analysis & Discussion

5.1 Paradigm Shift in Coding Approach

AI has changed the development workflow from imperative, syntax-oriented to architectural, declarative. The traditional Java method (Version A) involved a high cognitive load to manage the boilerplate code and manual memory handling, such as tracking array indices with a *bookCount* variable, managing a fixed-size *Book[]* array, and manually handling user inputs using Scanner loops. In the AI-assisted approach (Version B), the focus was on writing clear prompts and reviewing code. The AI automatically upgraded the rigid, fixed-size array structure into a dynamic collection, allowing the developer to focus on high-level system rules instead of handling array boundary restrictions.

5.2 Impact on Technical Comprehension

There is a clear trade-off in the effect of AI on learning Java. But on the other hand, the AI accelerated learning by instantly showing complex Java concepts. I asked it to add file handling, and it gave me complete working code using *BufferedReader* and *BufferedWriter* classes in Java, which was an instant learning reference. On the other hand, overreliance on AI can diminish deep understanding. The AI creates whole blocks at once, so a developer could bypass the critical trial-and-error stage of learning Java's checked exceptions (*IOException*), and only have a superficial familiarity with syntax, not a deep understanding of runtime execution.

5.3 Constraints and No-Go Zones

While AI is highly capable of generating standard Java structures, there are critical scenarios where it should not be utilized:

- Security-Critical Components: - AI tools should not be trusted blindly to write code handling user authentication, encryption algorithms, or sensitive database access layers, as they frequently suggest vulnerable or outdated Java library methods.

- **Complex Business Logic:-** In custom systems with intricate, proprietary business workflows, AI models often hallucinate logical conditions that violate system requirements.
- **Core Educational Foundations:-** Students should not use AI when learning foundational programming concepts such as manual primitive array manipulation, index boundaries, memory reference allocation, and core object-oriented encapsulation, as automation circumvents the cognitive development required to master these fundamentals.

5.4 Ethical and Legal Dimensions

The use of AI to help write code brings with it a host of tricky issues of authorship, plagiarism and intellectual property. Tools like ChatGPT and GitHub Copilot are trained on publicly available open-source repositories, so the Java code blocks they generate may sometimes inadvertently reproduce existing copyrighted code sequences without proper licensing attribution. Academic integrity is also compromised when students submit AI-generated logic structures as their own intellectual work. It's a reminder of the need for bright lines between legitimate developmental aid and robotic plagiarism.

6. Conclusion

6.1 Summary of Key Findings

This report explored how integrating AI tools affects the software development process, focusing directly on a Java-based environment. By building a Library Management System using two different methods, a clear contrast was observed. Version A (the manual approach) relied on standard primitive structures like a fixed-size *Book[]* array and direct *Scanner* inputs. While this required writing explicit logic and tracking array bounds manually, it offered complete control over the system mechanics.

On the other hand, Version B (the AI-assisted approach) demonstrated how automation can instantly upgrade a project. The AI quickly replaced the rigid array limits with a dynamic *ArrayList*, introduced file storage for data persistence, and applied robust *try-catch* blocks to prevent crashes. While AI tools significantly cut down development time and add advanced features easily, they can lead to over-dependency if developers do not understand the underlying Java fundamentals.

6.2 Key Takeaway

Ultimately, AI coding assistants are not replacements for strong programming skills, but rather useful extensions of them. In the software industry, tools like ChatGPT and GitHub Copilot act as excellent productivity boosters. They allow developers to skip tedious boilerplate coding and focus on system architecture, code refactoring, and rapid prototyping.

However, within academic programming education, AI tools must be used with caution. Students need to learn core foundational concepts—such as array manipulation, primitive indexing, error checking, and object encapsulation—manually first. Relying on AI too early shortcuts the critical trial-and-error phase needed to understand code execution. In conclusion, AI is a powerful tool to enhance development efficiency, but maintaining human critical thinking is essential to ensure code quality, security, and true technical understanding.

- GitHub Repository: <https://github.com/pasindusandeepa794/Library-Management-System>